Experiment 9 – Apply Minimax with Alpha-Beta Pruning on given problem

Learning Objective: To understand and implement the Minimax algorithm with Alpha-Beta Pruning for adversarial search problems, and to analyse how pruning techniques improve efficiency by reducing unnecessary node exploration while maintaining optimal decision-making in competitive environments using Python.

Tools: Visual Code Studio

Theory: -

In Artificial Intelligence, adversarial search algorithms are used in competitive environments where multiple agents with opposing goals interact. The Minimax algorithm is a fundamental approach used to determine optimal decisions in such environments by assuming that both players act rationally. However, Minimax can be computationally expensive as it explores the entire game tree. To overcome this limitation, Alpha-Beta Pruning is introduced as an optimization technique that reduces the number of nodes evaluated without affecting the final result.

This experiment focuses on the Minimax algorithm enhanced with Alpha-Beta Pruning for efficient decision-making in two-player games.

1. Recap of Minimax Algorithm
The Minimax algorithm constructs a game tree where:

- Each node represents a game state.
- MAX nodes aim to maximize the score.
- MIN nodes aim to minimize the score.
- Leaf nodes contain utility values (e.g., +1, 0, -1).

The algorithm explores all possible moves, evaluates terminal states, and propagates values upward to determine the optimal move.

2. Need for Alpha-Beta Pruning
The major drawback of the Minimax algorithm is its high time complexity, as it evaluates every node in the game tree. In large games, this becomes inefficient and impractical. Alpha-Beta Pruning improves performance by eliminating branches that cannot influence the final decision.

3. Alpha-Beta Pruning Concept
Alpha-Beta Pruning works by keeping track of two values during the search:

- Alpha (α): The best (maximum) value that the MAX player can guarantee so far.
- Beta (β): The best (minimum) value that the MIN player can guarantee so far.

The algorithm prunes (cuts off) branches when:

- At a MAX node, if the current value becomes greater than or equal to β, further exploration is unnecessary.
- At a MIN node, if the current value becomes less than or equal to α, further exploration is unnecessary.

This means that certain parts of the tree are ignored because they will not affect the final outcome.

4. Working of Alpha-Beta Pruning

- The algorithm performs a depth-first search similar to Minimax.
- At each node, it updates α and β values.
- When a pruning condition is met, the remaining child nodes are skipped.
- The final decision remains the same as standard Minimax but with fewer node evaluations.

5. Advantages of Alpha-Beta Pruning

- Significantly reduces the number of nodes explored.
- Improves time efficiency without affecting optimality.
- Allows deeper search within the same computational limits.

6. Properties of Minimax with Alpha-Beta Pruning

- Complete: Yes, if the game tree is finite.
- Optimal: Yes, it produces the same result as Minimax.
- Time Complexity: Best case reduces to O(b^(d/2)), depending on node ordering.
- Space Complexity: Similar to Minimax due to depth-first traversal.

7. Applications
Alpha-Beta Pruning is widely used in:

- Game AI (chess, checkers, tic-tac-toe)
- Decision-making systems in competitive environments
- Strategic planning and simulation problems

Overall, Alpha-Beta Pruning enhances the efficiency of the Minimax algorithm by reducing unnecessary computations, making it suitable for solving complex adversarial problems more effectively.

Fagir Singh Charitable Trust's (Regd.)
THAKUR COLLEGE OF ENGINEERING & TECHNOLOGY
ISO 9001 : 2015 Certified
NBA and NAAC Accredited

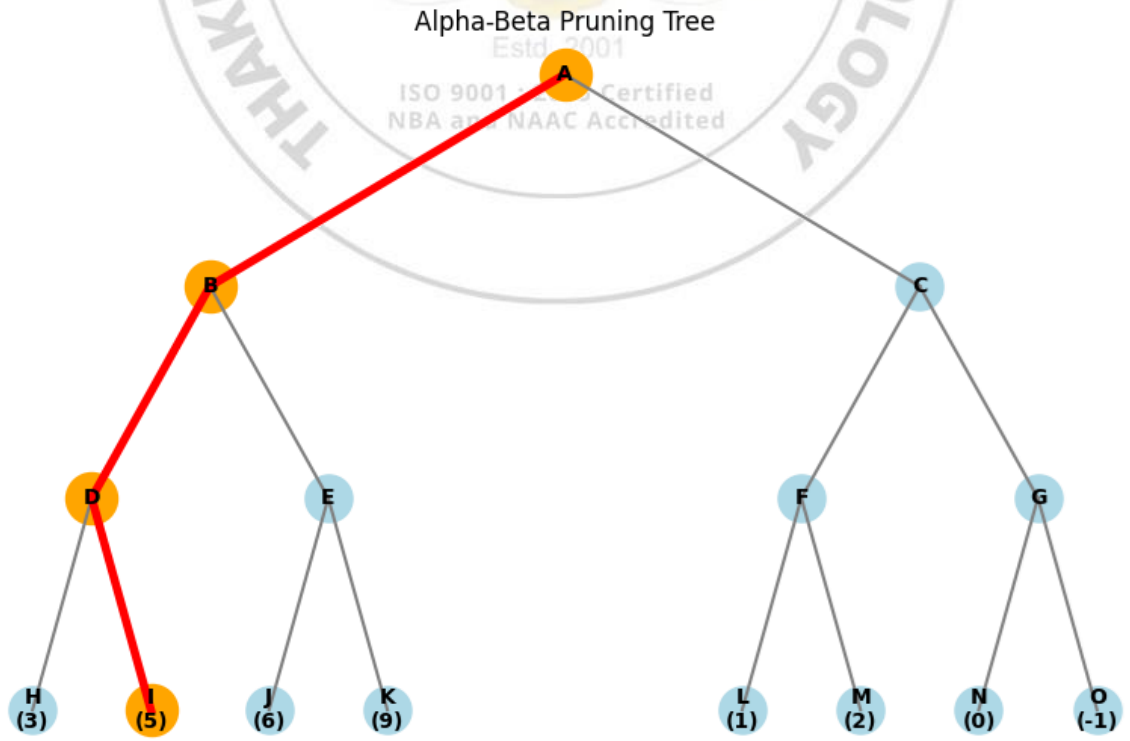
Code & Output: -

```
import matplotlib.pyplot as plt
# -------- TREE STRUCTURE --------
tree = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F', 'G'],
    'D': ['H', 'I'],
    'E': ['J', 'K'],
    'F': ['L', 'M'],
    'G': ['N', 'O'],
    'H': [], 'I': [], 'J': [], 'K': [],
    'L': [], 'M': [], 'N': [], 'O': []
}
# -------- LEAF VALUES --------
values = {
    'H': 3, 'I': 5,
    'J': 6, 'K': 9,
    'L': 1, 'M': 2,
    'N': 0, 'O': -1
}
# -------- NODE POSITIONS --------
pos = {
    'A': (0, 5),

    'B': (-3, 4), 'C': (3, 4),

    'D': (-4, 3), 'E': (-2, 3),
    'F': (2, 3), 'G': (4, 3),

    'H': (-4.5, 2), 'I': (-3.5, 2),
    'J': (-2.5, 2), 'K': (-1.5, 2),
    'L': (1.5, 2), 'M': (2.5, 2),
    'N': (3.5, 2), 'O': (4.5, 2)
}
# -------- ALPHA-BETA FUNCTION --------
def alphabeta(node, depth, alpha, beta, is_max):

    # Leaf node
    if node not in tree or len(tree[node]) == 0:
        return values[node], [node]

    if is_max:
        best_val = float('-inf')
        best_path = []

        for child in tree[node]:
            val, path = alphabeta(child, depth + 1,
alpha, beta, False)

            if val > best_val:
                best_val = val
                best_path = [node] + path

            alpha = max(alpha, best_val)

            # PRUNING CONDITION
            if beta <= alpha:
                break

        return best_val, best_path

    else:
        best_val = float('inf')
        best_path = []

        for child in tree[node]:
            val, path = alphabeta(child, depth + 1,
alpha, beta, True)

            if val < best_val:
                best_val = val
                best_path = [node] + path

            beta = min(beta, best_val)

            # PRUNING CONDITION
            if beta <= alpha:
                break

        return best_val, best_path
# -------- DRAW FUNCTION --------
def draw_tree(ax, title, path=None):

    # Draw edges
    for u in tree:
        for v in tree[u]:
            x1, y1 = pos[u]
            x2, y2 = pos[v]
            ax.plot([x1, x2], [y1, y2], color='gray')
```

```
    # Highlight optimal path
    if path:
        for i in range(len(path) - 1):
            x1, y1 = pos[path[i]]
            x2, y2 = pos[path[i+1]]
            ax.plot([x1, x2], [y1, y2], linewidth=4,
color='red')

    # Draw nodes
    for node, (x, y) in pos.items():
        if path and node in path:
            ax.scatter(x, y, s=600, color='orange')
        else:
            ax.scatter(x, y, s=500, color='lightblue')

        label = node
        if node in values:
            label += f"\n({values[node]})"

        ax.text(x, y, label, ha='center', va='center',
fontweight='bold')

    ax.set_title(title)
    ax.axis('off')
# -------- MAIN --------
optimal_value, optimal_path = alphabeta('A', 0,
float('-inf'), float('inf'), True)

print("Optimal Value:", optimal_value)
print("Optimal Path:", optimal_path)

# -------- PLOT --------
fig, ax = plt.subplots(figsize=(10, 6))
draw_tree(ax, "Alpha-Beta Pruning Tree",
optimal_path)
plt.show
```

Output: -

Optimal Value: 5
Optimal Path: ['A', 'B', 'D', 'I']



Learning Outcomes: The student should have the ability to:

LO 9.1: Apply the Minimax algorithm with Alpha-Beta Pruning to solve adversarial search problems efficiently.
LO 9.2: Understand and utilize alpha (α) and beta (β) values to eliminate unnecessary node exploration.
LO 9.3: Analyse the improvement in performance and efficiency compared to the basic Minimax algorithm.
LO 9.4: Implement optimized adversarial search techniques for game-based problem-solving using Python.

Course Outcomes:

Upon completion of this experiment, students will be able to understand and apply Genetic Algorithms to solve optimization problems, and analyse the effectiveness of evolutionary techniques in developing intelligent and adaptive solutions in Artificial Intelligence.

Conclusion:

Jagdu Singh Charitable Trust's (Regd.)
THAKUR COLLEGE OF ENGINEERING & TECHNOLOGY
Estd. 2001
ISO 9001 : 2015 Certified
NBA and NAAC Accredited

For Faculty Use: -

Correction Parameters	Formative Assessment [40%]	Timely completion of Practical [40%]	Attendance / Learning Attitude [20%]	
Marks Obtained				